

L'obiettivo del progetto è costruire un grafo degli attori e delle loro collaborazioni ed effettuare delle ricerche di cammini minimi su di esso. Per motivi didattici il progetto è suddiviso in parti di distinte da realizzare con linguaggi di programmazione diversi.

La correzione avverrà sulla macchina Linux laboratorio2.di.unipi.it al quale potete accedere tutti via ssh (con la [VPN](#) se da fuori dell'Università) utilizzando le credenziali universitarie. Non aspettate il giorno della consegna per provarla perché la macchina a volte è irraggiungibile. Lo spazio sul disco è limitato quindi dopo gli esperimenti cancellate i vostri dataset più grandi.

## CreaGrafo.java

Il primo passo del progetto consiste nel costruire il *grafo degli attori*, si tratta di un grafo non orientato in cui i nodi rappresentano attori o attrici ed esiste un arco tra l'attore *i* e l'attore *j* se essi hanno partecipato insieme in almeno un film o altro evento.

Tale grafo deve essere costruito a partire dai file name.basics.tsv e title.principals.tsv messi a disposizione da IMDB che sono file di testo di tipo tsv in cui le informazioni su ogni riga sono organizzate in campi suddivisi dal carattere \t. La prima riga di questi file indica l'informazione contenuta in ogni campo.

Il file name.basics.tsv contiene sei campi per ogni riga, le prime righe sono:

```
nconst    primaryName    birthYear    deathYear    primaryProfession    knownForTitles
nm0000001    Fred Astaire    1899        1987        actor,miscellaneous,producer
tt0072308,tt0050419,tt0027125,tt0031983
nm0000002    Lauren Bacall    1924        2014        actress,soundtrack,archive_footage
tt0037382,tt0075213,tt0117057,tt0038355
nm0000003    Brigitte Bardot    1934        \N        actress,music_department,producer
tt0057345,tt0049189,tt0056404,tt0054452
nm0000004    John Belushi    1949        1982        actor,writer,music_department
tt0072562,tt0077975,tt0080455,tt0078723
nm0000005    Ingmar Bergman    1918        2007        writer,director,actor
tt0050986,tt0069467,tt0050976,tt0083922
nm0000006    Ingrid Bergman    1915        1982        actress,producer,soundtrack
tt0034583,tt0038109,tt0036855,tt0038787
```

Il primo campo nconst è un stringa formata da nm seguito da 7 cifre; queste cifre sono diverse per ogni attore/attrice e quindi l'intero ottenuto da queste cifre costituirà il **codice identificativo univoco** di ogni attore/attrice.

Il secondo campo è il nome: questo non si può considerare un identificatore univoco in quanto esistono attori diversi con lo stesso nome. I due campi successivi sono l'anno di nascita e di morte: se questi non sono noti è presente la stringa \N. Il quinto campo contiene i ruoli avuti: si noti che non tutti sono attori, nell'elenco figurano anche registi, produttori, etc.

Il file title.principals.tsv contiene 6 campi su ogni riga ma noi interessano solamente il primo e il terzo. Il primo campo identifica un titolo e il terzo le persone che hanno lavorato per quel titolo. Ad esempio, le prime righe del file sono:

```
tconst    ordering    nconst    category    job    characters
tt0000001    1    nm1588970    self    \N    ["Self"]
tt0000001    2    nm0005690    director    \N    \N
```

|           |   |           |                 |                         |    |
|-----------|---|-----------|-----------------|-------------------------|----|
| tt0000001 | 3 | nm0005690 | producer        | producer                | \N |
| tt0000001 | 4 | nm0374658 | cinematographer | director of photography | \N |
| tt0000002 | 1 | nm0721526 | director        | \N                      | \N |
| tt0000002 | 2 | nm1335271 | composer        | \N                      | \N |
| tt0000003 | 1 | nm0721526 | director        | \N                      | \N |

Le quattro righe il cui primo campo è tt0000001 si riferiscono allo stesso titolo e nel terzo campo troviamo un elenco di persone che hanno lavorato a quel titolo, nel nostro caso sono le quattro persone nm1588970, nm0005690, nm0005690 e nm0374658. Le due righe successive si riferiscono al titolo tt0000001 e dal terzo campo otteniamo che a questo secondo titolo hanno lavorato le persone nm0721526 e nm1335271. Le informazioni su questo file sono quelle che ci permettono di stabilire se due attori hanno lavorato sullo stesso titolo (e quindi devono essere adiacenti nel grafo).

Il programma *CreaGrafo.java* deve leggere dalla linea di comando i nomi dei due file tsv visti sopra, elaborarli in maniera opportuna, e salvare il grafo degli attori. E' necessario creare una classe Attore con (almeno) le seguenti variabili di istanza:

```
int codice;           // codice univoco dell'attore
String nome;         // nome (e cognome) dell'attore
int anno;            // anno di nascita
Set<Integer> coprotagonisti; // codici degli attori che hanno recitato con this
```

oltre a qualsiasi metodo riteniate necessario. Il grafo degli autori deve essere costruito usando la classe Attore nel seguente modo:

- per prima cosa è necessario leggere il file name.basics.tsv e scartare tutti nomi che **contengono** \N nel campo data di nascita (birthYear) e che **non contengono** le parole actor o actress nel quinto campo (primaryProfession); in altre parole ci interessano solamente le persone che sono attori o attrici di cui è noto l'anno di nascita.
- per ogni nome rimasto si deve creare un'istanza di Attore inizializzando i suoi campi (a parte coprotagonisti) dai dati della rispettiva riga del file tsv: codice si ottiene dal primo campo eliminando le lettere nm e convertendolo in intero, nome e anno si ottengono dal secondo e terzo campo. Gli attori devono essere memorizzati in una variabile attori di tipo Map<Integer,Attore> che associa ad ogni codice l'istanza di Attore con quel codice
- il programma deve salvare sul file di testo nomi.txt il codice, il nome e l'anno di nascita di ogni attore in formato tsv: ogni riga del file deve contenere i dati di un singolo attore separati dal carattere \t. Gli attori devono essere scritti nel file in ordine di codice crescente. Il numero totale di righe deve essere 383965; per facilitarvi nel debug il file corretto nomi.txt è allegato al progetto, ma il vostro programma deve essere in grado di ricostruirlo.
- successivamente il programma deve leggere il file title.principals.tsv e utilizzare i dati per calcolare i coprotagonisti di ogni attore; per ogni titolo si deve costruire l'insieme cast delle persone che ci hanno lavorato e che appartengono alla collezione attori. Successivamente per ogni coppia a1, a2 di elementi in cast si deve aggiungere il codice di a1 alla variabile d'istanza coprotagonisti di a2 e il codice di a2 alla variabile d'istanza coprotagonisti di a1; si noti che non si deve mai aggiungere un attore fra i coprotagonisti di se stesso (quindi coprotagonisti di a1 non deve contenere il codice di a1)
- terminata l'elaborazione del file title.principals.tsv il programma deve scrivere il file grafo.txt considerando gli attori in ordine di codice crescente; per ogni attore deve

essere indicato il suo codice, il numero dei suoi coprotagonisti e l'elenco ordinato dei codici dei suoi coprotagonisti; i valori su una linea devono essere separati dal carattere \t. Dal punto di vista del grafo, questo elenco può essere visto come l'elenco dei nodi (un nodo per riga) con indicato per ogni nodo l'elenco dei nodi a cui è adiacente. Ad esempio, la linea: 101 6 2 5 9 22 159 405 indica che l'attore di codice 101 è adiacente ai 6 attori 2 5 9 22 159 405. Con i dati forniti, il grafo degli autori deve contenere 39959114 archi; il file 100grafo.txt contiene le prime 100 righe del file che rappresenta tale grafo.

- le istruzioni per la compilazione del programma java devono essere inserite nel makefile in modo che la compilazione avvenga scrivendo semplicemente `make` sulla linea di comando; successivamente il programma verrà testato con il comando `java CreaGrafo attori.tsv titoli.tsv` (con opportuni file .tsv). Se questa procedura non funziona (per qualsiasi motivo il programma non viene eseguito correttamente) il progetto sarà considerato insufficiente senza ulteriori indagini.

**Importante** l'algoritmo utilizzato per la costruzione del grafo deve leggere i due file di input una sola volta e deve utilizzare una strategia e strutture dati efficienti; data la dimensione dei dati non devono essere utilizzate strutture dati con operazioni di costo lineare. Il tempo di esecuzione di *CreaGrafo* deve essere inferiore ai 5 minuti, altrimenti l'esecuzione verrà interrotta e il progetto verrà considerato insufficiente.

## cammini.c

La seconda parte del progetto consiste in un programma C che legge i dati del grafo degli autori ed effettua il calcolo di cammini minimi. All'interno del programma C i nodi del grafo devono essere rappresentati dal tipo `attore` così definito:

```
typedef struct {
    int codice;    // codice attore
    char *nome;    // nome attore
    int anno;      // anno di nascita
    int numcop;    // numero coprotagonisti
    int *cop;      // array coprotagonisti
} attore;
```

dove `numcop` il numero di nodi adiacenti nel grafo, e `cop` è il puntatore un array di lunghezza `numcop` contenente i codici dei nodi adiacenti.

Se il grafo ha tota autori esso deve essere rappresentato da un array `attori` contenente tota oggetti di tipo `attore` ordinati per codice crescente, in altre parole deve essere `attori[i].codice < attori[i+1].codice` per  $i=0,\dots,tota-2$ . Questo è importante in quanto nel seguito ogni volta che dato un codice `c` si vuole accedere al relativo attore si deve effettuare una ricerca binaria sull'array `attori`; la ricerca binaria **deve** essere effettuata utilizzando la funzione di libreria `bsearch(3)`, consultare la pagina man per i dettagli.

L'eseguibile `cammini.out` deve essere invocato dalla linea di comando scrivendo

`cammini.out filenomi filegrafo numconsumatori`

dove *filenomi* e *filegrafo* devono essere i file *nomi.txt* e *grafo.txt* costruiti da *CreaGrafo* mentre *numconsumatori* deve essere un intero positivo che indica il numero di thread consumatori utilizzati durante la lettura del grafo (si veda nel seguito). Il grafo deve essere costruito mediante il seguente procedimento:

- per prima cosa si legge il file *nomi.txt* e si crea l'array attori iniziando solamente i campi codice, nome e anno di ogni attore
- successivamente si deve leggere il file *grafo.txt* utilizzando uno schema produttori consumatori. Il thread principale svolge il ruolo di produttore e legge le linee del file una alla volta con `getline(3)` e copia il puntatore alla linea letta su un buffer condiviso con i consumatori (il cui numero è dato dal parametro di input *numconsumatori*). I consumatori leggono le linee che vengono loro passate e utilizzano i dati in essa contenuti per inizializzare i campi *numcop* e *cop* di ogni attore. Dopo aver completato la lettura del file *grafo.txt* questi thread consumatori possono terminare

Terminata la lettura del grafo, il programma deve creare nella directory corrente una pipe *cammini.pipe* e aprirla in lettura. Successivamente il programma deve leggere dalla pipe due interi a 32 bit *a* e *b* e per ogni coppia di interi letta deve creare un nuovo thread che effettui il calcolo del cammino minimo tra *a* e *b* (si veda la sezione relativa per i dettagli).

Non appena la lettura della pipe non restituisce nulla (quindi l'altra estremità della pipe è stata chiusa in scrittura), il programma deve attendere 20 secondi (l'idea è di dare il tempo ad eventuali thread che calcolano i cammini minimi di completare il calcolo) e terminare deallocando tutte le strutture utilizzate dal programma.

Prima di ogni altra operazione il programma deve far partire un thread gestore di segnali che stampa il pid del processo su `stdout` e successivamente si mette in attesa con `sigwait` del segnale `sigint`. Se arriva il segnale `sigint` e il programma non è ancora nella fase lettura dalla pipe il thread deve stampare il messaggio *Costruzione del grafo in corso* e rimettersi in attesa con `sigwait`; se invece si è già nella fase di lettura dalla pipe, il programma deve terminare come descritto sopra (attendere 20 secondi e deallocare tutto) e il thread di gestione dei segnali stesso deve terminare.

Valgono le solite indicazioni che abbiamo visto durante il corso:

- non si devono mai usare variabili globali
- le allocazioni devono essere fatte dinamicamente mediante `malloc/realloc`
- in caso di errori durante l'esecuzione (input non valido, `malloc` fallita etc) il programma deve terminare con un messaggio esplicativo su `stderr` e con un exit code diverso da zero
- usate `stderr` per eventuali messaggi di debug e inviate a `stdout` solamente le informazioni indicate nel testo
- non scorrere mai l'array attori per cercare un singolo attore: usare la ricerca binaria
- tutta la memoria deve essere correttamente deallocata (verificare con `valgrind`)
- l'eseguibile *cammini.out* deve essere generato senza errori semplicemente dando il comando `make` dalla linea di comando (ricordate che `make` deve compilare contemporaneamente anche il programma java).

- dopo la fase di debugging compilate con l'opzione -O3 per velocizzare l'esecuzione e mettete questa opzione anche nel makefile in modo che venga utilizzata durante la correzione.

## Thread per il calcolo del cammino minimo

Questi thread ricevono come input il grafo (in pratica l'array attori) e il codice di due attori a e b e devono effettuare la ricerca di un cammino minimo da a a b all'interno del grafo utilizzando l'algoritmo [BFS](#) (in pratica è l'algoritmo di Dijkstra ottimizzato per il caso in cui gli archi hanno costo unitario). L'algoritmo utilizza una coda FIFO che potete implementare come ritenete opportuno (ma deve essere in grado di gestire un numero arbitrariamente grande di elementi, quindi un array di dimensione fissata non è accettabile). L'algoritmo deve anche tenere traccia dei nodi che sono già stati inseriti nella coda (variabile `explored` nello pseudocodice), a tale scopo **è obbligatorio** utilizzare un ABR (albero binario di ricerca). Questi thread devono essere resi `detached` (cioè non richiedere che venga fatta la `join`) con una opportuna chiamata alla funzione `pthread_detach(3)`

Ogni thread deve scrivere il proprio output sul file di nome `a.b` dove a e b sono i codici ricevuti in input. Ad esempio se il thread deve calcolare il cammino minimo tra i nodi 1000 e 2000, l'output deve essere scritto sul file `1000.2000` e deve consistere di uno dei seguenti messaggi:

1. codice 1000 non valido; se ad esempio il codice 1000 non corrisponde ad un attore nel grafo
2. non esistono cammini da 1000 a 2000; se i due codici corrispondono a due attori che non sono connessi nel grafo
3. se viene trovato un cammino minimo, l'output deve consistere degli attori che ne fanno parte secondo il seguente schema: per il cammino minimo di lunghezza 3 da 148 (Harrison Ford) a 8570840 (Zerocalcare) il file `148.8570840` dovrebbe contenere

```
148  Harrison Ford  1942
95   Woody Allen   1935
603090 Laura Morante 1956
8570840 Zerocalcare 1983
```

dove ogni riga contiene: codice, nome e anno di nascita di un attore separati dal carattere `\t`

Oltre a questo file di output, ogni thread deve stampare su `stdout` una riga di testo che descrive i codici ricevuti in input, l'esito della computazione, e l'indicazione del tempo di calcolo in secondi; ad esempio:

```
148.158: Lunghezza minima 3. Tempo di elaborazione 3.40 secondi
oppure
```

```
1000.2000: Nessun cammino. Tempo di elaborazione 0.01 secondi
```

Il tempo di elaborazione (`elapsed time`) può essere ottenuto mediante la funzione `times(3)`; potete vedere un esempio di uso nel programma `12atomic/stack.c`.

**NOTA IMPORTANTE:** dato che i nodi adiacenti ad un dato nodo sono ordinati per codice, il loro inserimento nell'ABR può creare dei cammini lunghi, che rallentano le operazioni. Per evitare questo fenomeno definite le funzioni

```
int shuffle(int n) {
    return (((n & 0x3F) << 26) | ((n >> 6) & 0x3FFFFFF)) ^ 0x55555555;
}
int unshuffle(int n) {
    return (((n >> 26) & 0x3F) | ((n & 0x3FFFFFF) << 6)) ^ 0x55555555;
}
```

e quando dovete inserire o cercare nell'ABR un codice *c* chiamate le funzioni di inserimento/ricerca passando il valore `shuffle(c)`. La funzione `unshuffle` è l'inversa di `shuffle` nel senso che `unshuffle(shuffle(i))=i`. Non dovrebbe servirvi ma non si sa mai. Usando `shuffle` una sequenza crescente viene trasformata in una sequenza molto meno regolare; ad esempio inserendo i 1570 coprotagonisti del primo attore si ottiene un ABR di altezza 34 aumentando quindi la velocità delle operazione sull'ABR di un fattore ~50.

## cammini.py

Questo programma lo fornisco io (lo trovate tra i file del progetto) ma dovete studiarne il codice per capirne il funzionamento e la sua relazione con `cammini.out` in modo da eventualmente discuterlo all'esame. Scrivete `cammini.py -h` sulla linea di comando per avere la descrizione del suo funzionamento.

A titolo di esempio, dopo aver lanciato `cammini.out` se si esegue `cammini.py 38 350125 3918035` dovrebbero essere generati i seguenti due file: `38.350125` con contenuto

```
38  Grace Kelly  1929
32  Charlton Heston 1923
324645 Loretta Goggi 1950
350125 Caterina Guzzanti 1976
e 350125.39180341 con contenuto:
```

```
350125 Caterina Guzzanti 1976
7746  Diego Abatantuono 1955
307   Helena Bonham Carter 1966
3918035 Zendaya 1996
```

Un commento sul parametro `-s`: ricordate che dal momento in cui viene chiusa la pipe il programma `cammini.out` e tutti i suoi thread terminano dopo 5 secondi; se in questo tempo i thread non hanno avuto il tempo di calcolare il cammino minimo questo non sarà calcolato e troverete il file `xxxx.yyyy` vuoto. Usando il parametro `-s` di `cammini.py` potete ritardare la chiusura delle pipe in `cammini.out` e quindi dare più tempo ai thread. Attenzione però che un tempo superiore ai 20 secondi per la ricerca di un cammino denota probabilmente un errore o l'uso di strutture dati non efficienti e quindi un progetto non sufficiente.

## Il progetto completo

Il materiale discusso fin'ora costituisce il progetto ridotto. Per il progetto completo si deve realizzare un programma che dati i codici di due attori fornisce l'elenco dei titoli in cui hanno lavorato assieme.

Si deve utilizzare il file `title.basics.tsv` che contiene 9 campi e le cui prime righe sono:

| tconst    | titleType | primaryTitle | originalTitle | isAdult | startYear | endYear | runtimeMinutes | genres            |
|-----------|-----------|--------------|---------------|---------|-----------|---------|----------------|-------------------|
| tt0000001 | short     | Carmencita   | Carmencita    | 0       | 1894      | \N      | 1              | Documentary,Short |

```

tt0000002 short Le clown et ses chiens Le clown et ses chiens 0 1892 \N 5 Animation,Short
tt0000003 short Poor Pierrot Pauvre Pierrot 0 1892 \N 5 Animation,Comedy,Romance
tt0000004 short Un bon bock Un bon bock 0 1892 \N 12 Animation,Short
tt0000005 short Blacksmith Scene Blacksmith Scene 0 1893 \N 1 Short
tt0000006 short Chinese Opium Den Chinese Opium Den 0 1894 \N 1 Short

```

A noi interessano solamente il primo campo, che identifica il titolo e il quarto campo che contiene il titolo originale del film/serie/etc. Osserviamo che i titoli sono rappresentati da un stringa formata da tt seguito da 7 cifre; queste cifre sono diverse per ogni titolo e quindi useremo l'intero ottenuto da queste cifre come codice identificativo del titolo in maniera simile a quanto visto per gli attori.

Fatta questa premessa per il progetto completo si devono realizzare le seguenti due componenti

## Costruzione elenco dei titoli

Modificare *CreaGrafo.java* in modo che venga creato anche un file partecipazioni.txt che per ogni attore elenchi i codici dei titoli in cui l'attore ha partecipato. Il file deve contenere una riga per ogni attore in ordine di codice attore crescente; per ogni attore deve essere indicato oltre al suo codice, il numero di titoli in cui ha partecipato e l'elenco ordinato dei codici dei titoli; come al solito i valori su una linea devono essere separati dal carattere \t. Ad esempio la riga corrispondente a Massimo Troisi (codice 873385) deve avere il seguente contenuto

```

873385 34 81427 84416 84646 87814 92635 93217 94263 96160 97051 103203 105120 110877
856243 3751272 4332660 5486932 7462118 9065566 9298998 9359168 11026682 12214720
12340624 12340858 12340862 12340868 12340880 12340902 12340918 12341064
12770004 12789532 21265604 26008250

```

da cui risulta che Massimo Troisi appare in 34 titoli, il primo dei quali ha codice 81427 che corrisponde al titolo tt0081427 (Ricomincio da tre). Il file completo deve avere 383965 righe e una dimensione di circa 216 Mb.

## collaborazioni.py

Questo programma Python deve prendere in input i file partecipazioni.txt e title.basics.tsv e una sequenza di codici di identificativi di attori *i1 i2 ... ik* e visualizzare su stdout l'elenco dei titoli in cui *i1* e *i2* hanno lavorato insieme, l'elenco in cui *i2* e *i3* hanno lavorato insieme e così via. Siete liberi di organizzare il codice come volete, con i seguenti vincoli:

1. i file di input devono essere letti una volta sola
2. dovete utilizzare strutture dati efficienti, in particolare dopo la lettura dei file non ci devono essere scansioni lineari dell'elenco degli attori o dei titoli

Esempio di un possibile output (cercate di mantenere questo formato) che fa riferimento all'ultimo esempio della sezione *cammini.py*

```

$ collaborazioni.py partecipazioni.txt title.basics.tsv 350125 7746 307 3918035
350125.7746: 1 collaborazioni:
3358470 Soap Opera

```

```

7746.307: 1 collaborazioni:
3109094 Dai nostri inviati: La Rai racconta la Mostra del cinema di Venezia 1980-1989

```

307.3918035: 1 collaborazioni:  
12943814 Drama Actress

=== Fine

Nel caso in cui una coppia non abbia collaborazioni va bene un output del tipo:

\$ collaborazioni.py partecipazioni.txt title.basics.tsv 3918035 95 603090  
3918035.95 nessuna collaborazione

95.603090: 1 collaborazioni:  
425817 Ceremonia de apertura del festival de cine de San Sebastián

=== Fine

Come vedete i titoli non sono soltanto film, ma spettacoli di ogni tipo (ma questo al vostro programma non interessa).